

Constraint Optimization through RTL Enhancement Using Generative AI

Team Timing Blinders — Shubhanshu Shivhare, Nagarjun BV

Astera Labs Nebula 2026, Digital Track

Abstract

A physical design engineer closing timing on a block spends most of the cycle doing the same four things: read the STA report, find the path that missed, pick a fix — pipeline it, retime a register, restructure the logic, or re-encode an FSM — edit the RTL by hand, and re-synthesize to see if it actually helped. None of that decision loop is inherently manual; it is manual because nobody has trusted an automated system enough to run it unsupervised. We build that system, and put the trust question first instead of last.

Three tools, one loop. `run_sta` wraps OpenSTA and turns a raw timing report into ranked critical paths with slack, in a form an LLM can actually read. `propose_and_apply_edit` hands that path, and only that path's RTL, to the model, constrained to emit exactly one of the four techniques above as edited Verilog — never prose, never a suggestion the model then has to also implement correctly by hand. `resynth_and_check` re-synthesizes the edit in Yosys, re-runs OpenSTA to confirm slack actually improved, and then — before the edit is allowed anywhere near floorplanning — checks it against the original RTL with EQY/SymbiYosys. Fail equivalence and the edit is discarded, no exceptions, no bypass. Papers in this space (ViTAD, SymRTLO, TimingLLM) get as far as diagnosing a violation and proposing a fix; none of them formally prove the fix didn't break the design before calling it a result. That gap is where this project sits.

The benchmark is sized to make the loop earn its numbers, not a toy that happens to synthesize cleanly: five independent asynchronous master clock domains, at least one generated clock per master, clock-domain crossings between them, clock dividers at more than one ratio, roughly 50,000 standard cells post-synthesis. It runs end to end through OpenROAD-flow-scripts on sky130hd, so every accepted edit ships with matched before/after timing, frequency, and PPA numbers alongside its equivalence-check trail — not just a slack delta.

Results get reported as measured, not as hoped for: how many of the injected critical paths close inside a fixed iteration budget, how much slack comes back on the ones that do, and which ones don't converge at all. ViTAD's 73.68% repair rate against a 54.38% plain-LLM baseline is the number we compare against, honestly, rather than around.